1. Activity Life Cycle (State Callbacks)

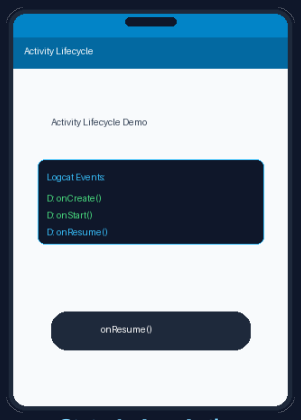
Concept: Monitoring application states as the user enters, leaves, and resumes the app.

■ ***Story — The Life of an Actor on Stage:***
- ***onCreate():*** *Born & dressed backstage (Load layout).*
- ***onStart():*** *Stage curtains open (Activity visible on screen).*
- ***onResume():*** *Spotlight on! Performing live (User interacts with UI).*
- ***onPause():*** *A quick interruption; phone rings (Another activity in front).*
- ***onStop():*** *Curtains close (App goes to background/home).*
- ***onRestart():*** *Waiting backstage before coming back.*
- ***onDestroy():*** *Show is over forever; actor goes home (Freed from memory).*

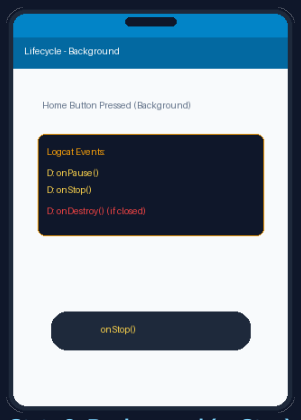
```
package com.example.exam;

import android.os.Bundle;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Toast.makeText(this, "onCreate()", Toast.LENGTH_SHORT).show();
    }
    @Override protected void onStart() {
        super.onStart();
        Toast.makeText(this, "onStart()", Toast.LENGTH_SHORT).show();
    }
    @Override protected void onResume() {
        super.onResume();
        Toast.makeText(this, "onResume()", Toast.LENGTH_SHORT).show();
    }
    @Override protected void onPause() {
        super.onPause();
        Toast.makeText(this, "onPause()", Toast.LENGTH_SHORT).show();
    }
    @Override protected void onStop() {
        super.onStop();
        Toast.makeText(this, "onStop()", Toast.LENGTH_SHORT).show();
    }
    @Override protected void onRestart() {
        super.onRestart();
        Toast.makeText(this, "onRestart()", Toast.LENGTH_SHORT).show();
    }
    @Override protected void onDestroy() {
        super.onDestroy();
        Toast.makeText(this, "onDestroy()", Toast.LENGTH_SHORT).show();
    }
}
```



State 1: App Active (onResume)



State 2: Background (onStop)

■ **Mnemonic: C-S-R-P-S-R-D → Come Soon, Run Promptly, Stop Running Dude! Every method must call super.onXxx().**

2. Registration, Login & SharedPreferences (3 Separate Views)

Concept: Multi-screen user registration, persistent key-value storage, and validation login.

■ ***Story — The Secret Diary & The Bouncer:***
- ***Registration Screen:*** *Write your secret password with a pen (edit().putString()), snap it shut (apply()).*
- ***SharedPreferences File:*** *The locked diary stored in phone's private storage (MODE_PRIVATE).*
- ***Login Screen:*** *The Bouncer opens the diary (getString()), checks your password, and says 'Welcome!'.*

```
// 1. REGISTER ACTIVITY (MainActivity.java)
package com.example.exam;

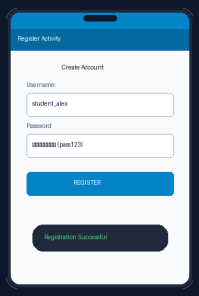
btn.setOnClickListener(v -> {
    SharedPreferences sp = getSharedPreferences("MyPrefs", MODE_PRIVATE);
    sp.edit()
      .putString("U", u.getText().toString())
      .putString("P", p.getText().toString())
      .apply();

    Toast.makeText(this, "Registered!", Toast.LENGTH_SHORT).show();
    startActivity(new Intent(this, LoginActivity.class));
});

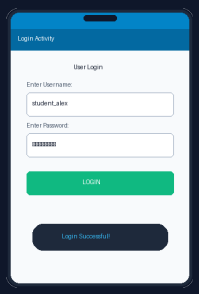
// 2. LOGIN ACTIVITY (LoginActivity.java)
package com.example.exam;

btn.setOnClickListener(v -> {
    SharedPreferences sp = getSharedPreferences("MyPrefs", MODE_PRIVATE);
    String savedU = sp.getString("U", "");
    String savedP = sp.getString("P", "");

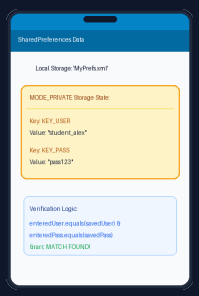
    if (u.getText().toString().equals(savedU) &&
        p.getText().toString().equals(savedP)) {
        Toast.makeText(this, "Login Success", Toast.LENGTH_SHORT).show();
    } else {
        Toast.makeText(this, "Login Failed", Toast.LENGTH_SHORT).show();
    }
});
```



1. Register Screen



2. Login Screen



3. SharedPrefs Storage

■ **Formula: Get → Edit → Put → Apply to write | Get → getString() to read.**

3. Toggle Image (Dynamic View Switching)

Concept: Toggling between two distinct image drawable resources upon button click.

■ ***Story — The Two-Way Light Switch:***
• Think of a wall switch (boolean isFirstImage = true;).
• Flip the switch once → Sun sets, Moon rises (setImageResource(R.drawable.image2)) and mark switch down (false).
• Flip it again → Moon sets, Sun rises (setImageResource(R.drawable.image1)) and mark switch up (true).

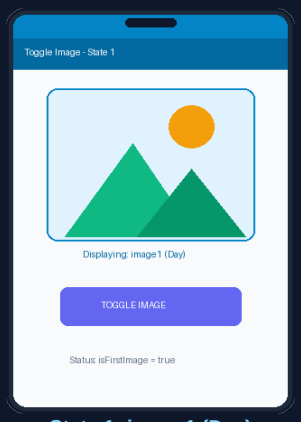
```
package com.example.exam;

public class MainActivity extends AppCompatActivity {
    private boolean isFirstImage = true;

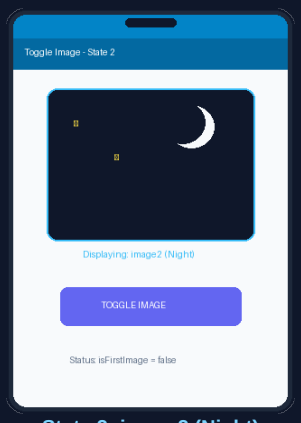
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        ImageView imageView = findViewById(R.id.imageView);
        Button btnToggle = findViewById(R.id.btnToggle);

        btnToggle.setOnClickListener(v -> {
            if (isFirstImage) {
                imageView.setImageResource(R.drawable.image2);
                isFirstImage = false;
            } else {
                imageView.setImageResource(R.drawable.image1);
                isFirstImage = true;
            }
        });
    }
}
```



State 1: image1 (Day)



State 2: image2 (Night)

■ **Core Method: imageView.setImageResource(R.drawable.image_name); accompanied by inverting the boolean flag.**

4. Spinner Component (Dropdown Selection)

Concept: Populating and handling selection events on a dropdown Spinner using ArrayAdapter.

■ ***Story — The Restaurant Waiter:***
- ***String[] array:*** *The dishes prepared in the kitchen (Data Source).*
- ***Spinner:*** *The customer's table (UI View).*
- ***ArrayAdapter:*** *The Waiter who bridges the kitchen to the table.*
- ***onItemSelected():*** *The customer taps a dish, and the waiter serves it with a Toast.*

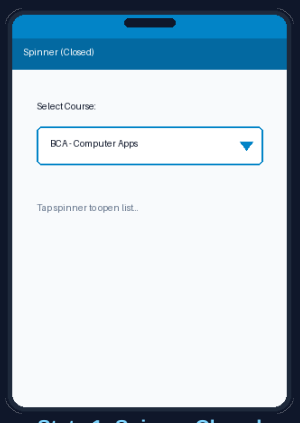
```
package com.example.exam;

public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle b) {
        super.onCreate(b);
        setContentView(R.layout.activity_main);

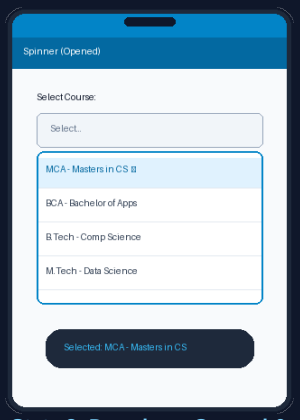
        Spinner s = findViewById(R.id.spinner);
        String[] c = {"MCA", "BCA", "B.Tech", "M.Tech"};

        // 1. One-line Adapter setup
        s.setAdapter(new ArrayAdapter<>(this,
            android.R.layout.simple_spinner_dropdown_item, c));

        // 2. Selection Listener
        s.setOnItemSelectedListener(new AdapterView.OnItemSelectedListener() {
            @Override
            public void onItemSelected(AdapterView<?> p, View v, int pos, long id) {
                Toast.makeText(MainActivity.this, c[pos], Toast.LENGTH_SHORT).show();
            }
            @Override public void onNothingSelected(AdapterView<?> parent) {}
        });
    }
}
```



State 1: Spinner Closed



State 2: Dropdown Opened & Chosen

■ **Spinner Flow: Array → ArrayAdapter → setAdapter → setOnItemSelectedListener.**

5. ListView with ArrayAdapter (List Scrolling & Item Clicks)

Concept: Binding an array of strings to a scrollable ListView and detecting clicked row positions.

■ ***Story — The Supermarket Fruit Shelves:***
- ***String[] fruits:*** *Fruit boxes sitting in the store inventory.*
- ***ListView:*** *The vertical shopping shelves displayed to customers.*
- ***ArrayAdapter:*** *The shelf-stacker who puts each fruit on a uniform plate (simple_list_item_1).*
- ***onItemClick():*** *Shopper touches a fruit, and the system announces what was chosen.*

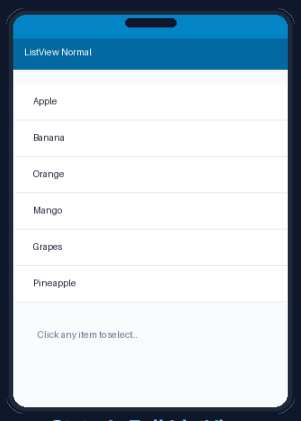
```
package com.example.exam;

public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle b) {
        super.onCreate(b);
        setContentView(R.layout.activity_main);

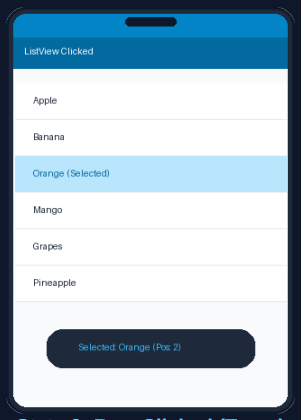
        ListView lv = findViewById(R.id.listView);
        String[] fruits = {"Apple", "Banana", "Orange", "Mango", "Grapes"};

        // 1. Adapter bridges String array -> ListView rows
        lv.setAdapter(new ArrayAdapter<>(this,
            android.R.layout.simple_list_item_1, fruits));

        // 2. Row Click Listener
        lv.setOnItemClickListener((p, v, pos, id) ->
            Toast.makeText(this, fruits[pos], Toast.LENGTH_SHORT).show()
        );
    }
}
```



State 1: Full ListView



State 2: Row Clicked (Toast)

■ **Key Difference: ListView uses setOnItemClickListener() while Spinner uses setOnItemSelectedListener().**

6. Explicit Intent (Screen-to-Screen Navigation & Data Passing)

Concept: Directly launching a specific target activity and transferring string extras between them.

■ ***Story — The VIP Courier with Named Envelope:***
- ***Envelope (Intent):*** *Addressed directly from this to SecondActivity.class.*
- ***putExtra("KEY", val):*** *Slipping the secret message into the envelope before mailing.*
- ***startActivity():*** *Dispatching the courier off to deliver.*
- ***getIntent().getStringExtra():*** *Opening the envelope at House #2 to read the message.*

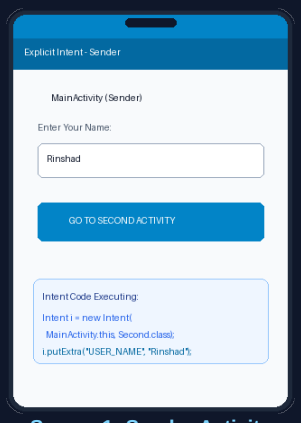
```
package com.example.exam;

// 1. SENDER ACTIVITY (MainActivity.java)
btn.setOnClickListener(v -> {
    Intent i = new Intent(this, SecondActivity.class);
    i.putExtra("USER", et.getText().toString());
    startActivity(i);
});

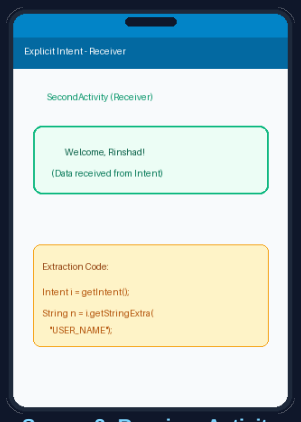
// 2. RECEIVER ACTIVITY (SecondActivity.java)
@Override
protected void onCreate(Bundle b) {
    super.onCreate(b);
    setContentView(R.layout.activity_second);

    TextView tv = findViewById(R.id.tvWelcome);
    String name = getIntent().getStringExtra("USER");
    tv.setText("Welcome, " + name + "!");
}

// 3. AndroidManifest.xml:
// <activity android:name=".SecondActivity" />
```



Screen 1: Sender Activity



Screen 2: Receiver Activity

■ **Sender 3 P's: Prepare new Intent(...) → Pack putExtra(...) → Push startActivity().**
Receiver 2 G's: Get getIntent().getStringExtra() → Give to setText().